CrimeGlass v1 - Summary

Here is the expanded, highly detailed technical summary covering every feature, algorithm, pipeline step, mathematical formulation, library, module, and database model engineered throughout our conversation.

Detailed Architecture & Technical Manual

1. Technical Libraries & Module Stack Inventory

Backend & Web Server Layer

- `fastapi` (v0.100+): Primary web engine handling CORS, background execution, schema validation, and REST/WebSocket routing.
- `uvicorn`: ASGI web server enabling high-concurrency asynchronous socket connections.
- `pydantic` (v2.x): Strict typing and validation models (`GraphNode`, `GraphEdge`, `SuspectProfile`, `DashboardMetricsResponse`, `HypergraphResponse`, `MastermindResponse`).
- `contextlib.asynccontextmanager`: Manages lifespan pool allocation and graceful shutdown for the `AsyncGraphDatabase` Bolt connection pool.
- `fastapi.WebSocket` & `WebSocketDisconnect`: Implements the `ConnectionManager` class for multi-investigator room management, broadcasting real-time progress (`PIPELINE_PROGRESS`) and graph updates (`GRAPH_UPDATED`).

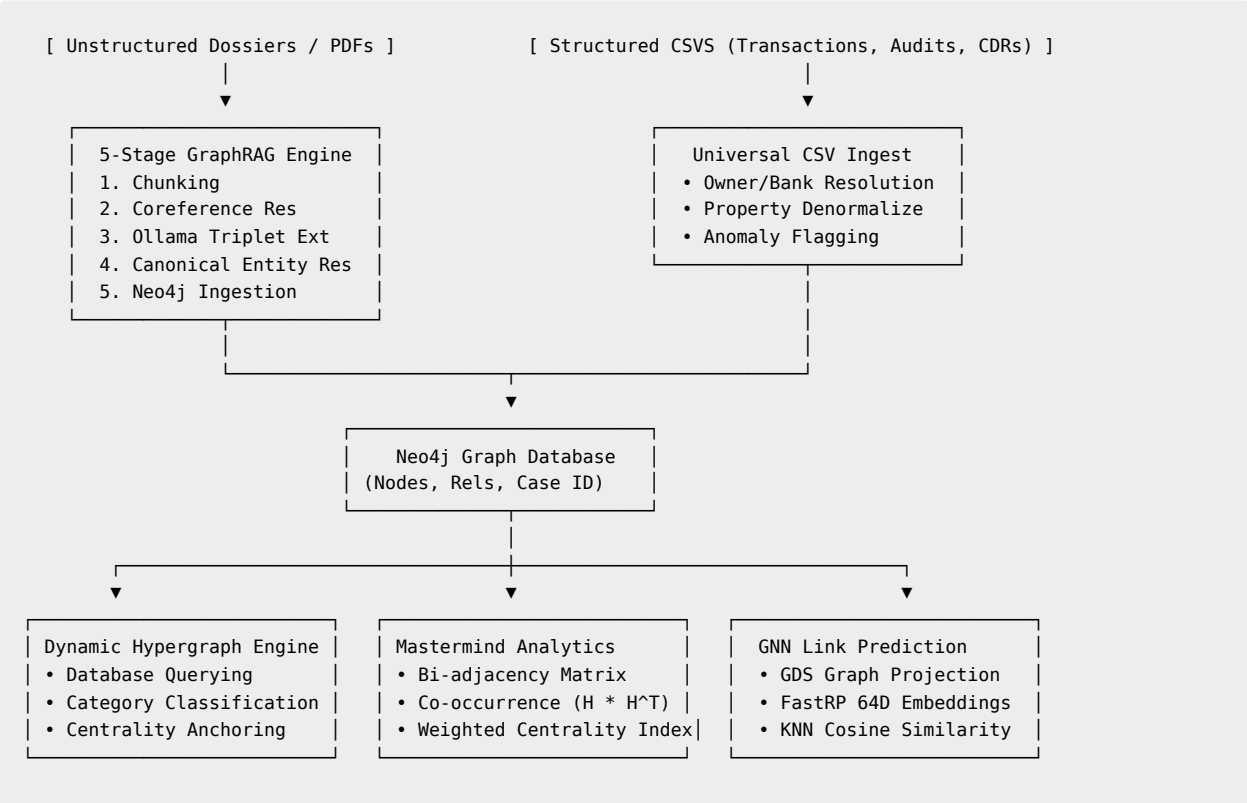
Graph Database & Graph Science Frameworks

- `neo4j.AsyncGraphDatabase` & `AsyncDriver`: Asynchronous Python driver executing parameter-bound Cypher queries.
- `Neo4j Graph Data Science (GDS)`: In-memory C++ graph engine executing `gds.graph.project.cypher`, `gds.fastRP.mutate`, and `gds.knn.write`.
- `networkx` & `networkx.algorithms.bipartite`: In-memory graph processing engine used to project bipartite hypergraph structures (Entity-Event) into co-occurrence matrices.
- `sklearn.preprocessing.StandardScaler`: Normalizes disparate centrality metric distributions ($Z = \frac{x - \mu}{\sigma}$) prior to weighted sum aggregation.
- `numpy`: Handles bi-adjacency matrix calculations, linear algebra dot products ($H \cdot H^T$), and zeroing diagonal self-loops.
- `pandas`: Dataframe manipulation engine for merging CSV entity datasets, handling missing values, and formatting dates.

NLP, LLM Extraction & Cryptography

- `ollama`: Local LLM interface connecting to `qwen2.5` (triplet extraction) and `gemma2:9b` (natural language insight parsing into Cypher mutations).
- `hashlib`: Executes `sha256()` hashing algorithms for document content and cryptographic blockchain block headers.
- `re` & `json`: Regex-based header splitting, token cleaning, and structured JSON parsing.

2. End-to-End Processing Flow & Feature Map



A. Unstructured Ingestion Pipeline (5 Steps)

1. **Step 1: Chunking (`step1_chunking.py`):** Splits files into overlapping narrative blocks while capturing structural section titles.
2. **Step 2: Coreference Resolution (`step2_coref.py`):** Resolves ambiguous pronouns (*he*, *they*, *the subject*) across adjacent sentences to preserve subject identity.
3. **Step 3: LLM Triplet Extraction (`step3_extraction.py`):** Invokes Ollama to extract explicit `(Subject, Predicate, Object)` triplets with verbatim evidence quotes.
4. **Step 4: Canonical Entity Resolution (`step4_resolution.py`):** Maps aliases (e.g., *"The Big Bull"*, *"Harshad Shantilal Mehta"*) to a single canonical node (*"Harshad Mehta"*).
5. **Step 5: Neo4j Ingestion (`step5_ingest.py`):** Creates `:Entity`, `:Case`, and `:Document` nodes with `BELONGS_TO` ownership edges.

B. Universal CSV Ingestion (`csv_processor.py`)

- **Property Denormalization:** Joins accounts (`A001A050`) directly with human owners (`P001` - Harshad Mehta) and banks (`B006` - Deccan Banking Corp) upon ingestion.
- **Currency & Timestamp Standardization:** Standardizes amounts (converts Crores to raw INR numbers) and maps timestamps to ISO formats.

C. Cryptographic Blockchain Verification

- **Table Conversion (`convert_dossier_to_table`):** Parses raw text dossiers into section records (`SEC_PREAMBLE`, `SECTION_1`, etc.).
- **Hash Chain Assembly (`generate_blockchain_hash_table`):** Builds a cryptographic audit ledger where each block contains `Index | Timestamp | Record ID | Data Hash | Previous Hash`.
- **Tamper Verification (`verify_blockchain_integrity`):** Validates payload and pointer consistency across the chain.

3. Hypergraph Engine & Mastermind Centrality Formulation

Database-Driven On-Demand Hypergraph Assembly

Rather than relying on local JSON files, the hypergraph engine queries Neo4j directly across both text-extracted and CSV-ingested nodes. It groups entities by context/chunk into hyper-edges and categorizes them into six universal operational domains:

- `Communication`, `Financial_Transaction`, `Violent_Crime`, `Market_Conspiracy`, `Legal_Action`, `Physical_Meeting`.

Mastermind Centrality Mathematics

1. **Bi-Adjacency Matrix (H):** Formed between M people nodes and N event nodes, where $H_{i,j} = 1$ if person i participated in event j.
2. **Co-occurrence Matrix (C):** $C = H \cdot H^T$, where diagonal elements $C_{i,i}$ are set to 0.
3. **Graph Projection:** Projected into a weighted NetworkX core graph G_{core}.
4. **Weighted Centrality Index:** Metrics are standardized (Z-scores) and combined via a weighted sum:

   $$\text{Mastermind Index} = 0.40 \cdot Z(\text{Betweenness}) + 0.35 \cdot Z(\text{PageRank}) + 0.15 \cdot Z(\text{Degree}) + 0.10 \cdot Z(\text{Closeness})$$

4. GNN Link Prediction (FastRP + KNN)

1. **Cypher In-Memory Projection:**Cypher

```
CALL gds.graph.project.cypher(
  'gnn_workspace',
  'MATCH (n {case_id: $case_id}) RETURN id(n) AS id',
  'MATCH (n {case_id: $case_id})-[r]-(m {case_id: $case_id}) RETURN id(n) AS source, id(m) AS target',
  {parameters: {case_id: $case_id}}
)
```

2. **Fast Random Projection (FastRP):** Generates 64-dimensional dense structural vector embeddings for each node.
3. **K-Nearest Neighbors (KNN):** Evaluates cosine distance between node embedding pairs to predict hidden relationships (`INFERRED_ASSOCIATE`).
4. **Post-Processing Cleanup:** Deletes self-loops and pre-existing edges to present only novel predictions.